

Relatório — Fase 5 (concluída e validada): Proveniência OKF + Attested Computation (Inserção F)

Projeto: LangNet · caso-teste ClinIA · **Data:** 2026-08-13 · **Executor:** Claude **Commit selo:** 507b02f · **Checkpoint de rollback (antes da fase):** cc6f806

A Fase 5 grava **proveniência/confiança/atualidade** no **vocabulário OKF v0.2** e emite conceitos **Attested Computation** para as tasks — reconhecendo nossos adapters determinísticos + contrato (A) + verificação (B) no **padrão OKF**. Enriquece o bundle (conhecimento); **não altera telas nem o runtime de execução**.

1. O que foi implementado

No gerador (backend/agents/langnetagents.py): - `_okf_provenance_fm` + `_emit_okf_bundle` **enriquecido**: cada conceito de **tabela** ganha frontmatter OKF v0.2: **sources** (artefato-fonte), **generated: {by, at}** (convenção de ator langnet/<modelo>), **status**, e **verified: [{by, at}]** com human:<id> quando a fonte foi **aprovada** → deriva o **trust tier** (unverified / human-reviewed). - **Novos conceitos knowledge/tasks/<task>.md com type: Attested Computation**: runtime, "Computação sancionada" (adapters:<task>_deterministic ; o agente só fornece parâmetros, não edita a computação), **Receipt** (= output_schema da Fase 1) e **Attester** (= verification da Fase 4). - **Code-gen**: passa generated_by (modelo), generated_at, verified_by (se o Modelo de Dados foi aprovado) e source_ref. Query do data model agora inclui status / approved_by.

2. Provas de validação (saídas reais)

2.1 Proveniência OKF no conceito de tabela (tables/pre_diagnostics.md)

```
type: DB Table
title: pre_diagnostics
status: stable
sources:
  - resource: data-model://a3ae2f89-...
generated:
  by: langnet/qwen2.5-coder-32b-instruct
  at: 2026-08-13T21:50:24
```

2.2 Conceito Attested Computation (tasks/criar_encaminhamento.md)

```
---
type: Attested Computation
title: criar_encaminhamento
runtime: mysql
generated: { by: langnet/qwen2.5-coder-32b-instruct, at: ... }
---
# Computação sancionada
Executada pela camada determinística (adapters:criar_encaminhamento_deterministic).
O agente PODE apenas fornecer valores para os parâmetros; NÃO edita a computação.
# Receipt (contrato de saída)
- Propriedades: encaminhamento_id, status
# Attester (verificação)
```

```
- require_inputs: atendimento_id
- row_check: linha em [encaminhamentos](/tables/encaminhamentos.md) com FKs de contexto
['atendimento_id']
```

✓ 12 conceitos Attested Computation gerados — nossos **adapters + contrato (A) + verificação (B)** no **padrão OKF**.

2.3 Trust tier (unverified × human-reviewed)

```
ClinIA (Modelo de Dados = draft)      -> verified AUSENTE  => tier: unverified   (honesto)
demo com verified_by=human:pasteur    -> verified:[{by: human:pasteur}] => tier: human-reviewed
```

✓ O `verified` só aparece quando há **aprovação humana** (o nosso passo "Aprovar" = `verified humano`).

2.4 E2E (`./smoke.sh`) — regressão zero

```
[verify] atendimento=True | encaminhamento(medico+esp)=True | prontuario(liga pre_diag+enc)=True
[smoke]      VERDE
```

✓ O ws-server lê o bundle enriquecido normalmente; o fluxo clínico completo continua persistindo a cadeia.

3. Telas

A Fase 5 não altera nenhuma tela (nem o runtime de execução) — enriquece o **conhecimento** (bundle OKF). O smoke seguiu **VERDE**.

4. Benefício e trilha de commits

- **Benefício:** rastreabilidade **padrão e portátil** ("gerado por `langnet/qwen...`, verificado por `human:...`, `sources`, `status`", legível por agente **e humano**); e o que inventamos (adapter sancionado receipt + attester) ganha o **nome padrão do OKF** (Attested Computation).
- **Rollback:** `cc6f806` (*ANTES da Fase 5*). **Selo:** `507b02f`.

5. Próximo passo

Fase 6 — gate de requisito + auto-crítica (Inserção C): impor os 8 elementos na Especificação/Agent-Task Spec, um **gate** que barra spec incompleta e um passo de **auto-crítica** (à la RLAIIF). Será precedida pelo **CHECKPOINT — ANTES da Fase 6**.